

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FCCD – Software



Tarea:

Práctica Experimental Unidad 2

Materia:

Aplicaciones Web

Integrantes:

Álava Alvarado Jean Pierre
Rivera Suárez Jonathan Javier
Tejada Bazaña Luis Alejandro

Docente:

Ing. Guerrero Ulloa Gleiston Ciceron

Curso:

5to Software “A”

Quevedo – Los Ríos – Ecuador

19/06/2026

Índice

1. Introducción	1
2. Desarrollo	2
2.1. Fundamento teórico	2
2.1.1. Tema 1: Introducción a la programación del lado del servidor	2
2.1.2. Tema 2: Desarrollo web con Java y JSP	6
2.1.3. Tema 3: Desarrollo web con ASP.NET	8
2.2. Descripción general del sistema	11
2.3. Tecnologías utilizadas	11
2.4. Funcionalidades implementadas	12
2.5. Controles de seguridad implementados	13
2.5.1. Prevención de Cross-Site Scripting	13
2.5.2. Protección contra ataques CSRF	13
2.5.3. Almacenamiento seguro de contraseñas	14
2.5.4. Cabeceras HTTP de seguridad	15
2.5.5. Prevención de inyección SQL	15
3. Resultados obtenidos	16
3.1. Resultados de la implementación en PHP	16
3.2. Resultados de la implementación en Java con Spring Boot	20
3.3. Resultado del análisis estático con PHPStan	23
4. Comparación entre PHP y Java con Spring Boot	24
5. Análisis de la práctica	26
5.1. Tecnología con mayor seguridad por defecto	26
5.2. Tecnología recomendada para el contexto del proyecto	26
5.3. Resultado obtenido mediante PHPStan	26
5.4. Integración con el frontend desarrollado en la Unidad 1	27
5.5. Flujo de la primera petición autenticada	27
6. Conclusiones	29
7. Repositorio del proyecto	30
Referencias bibliográficas en formato IEEE	31
Referencias bibliográficas en formato APA	33

1. Introducción

El desarrollo web del lado del servidor constituye una parte fundamental en la construcción de aplicaciones dinámicas, debido a que permite procesar solicitudes, gestionar usuarios, controlar el acceso a los recursos y establecer comunicación con una base de datos. A diferencia de una página estática, una aplicación web dinámica puede generar respuestas según la información almacenada, las acciones realizadas y los permisos asignados a cada usuario.

En la presente práctica experimental se desarrolló el sistema SGROAS, correspondiente a un Sistema de Gestión de Recursos Operativos. La aplicación permite administrar la información de conductores mediante un módulo CRUD, además de incorporar un sistema completo de autenticación compuesto por registro de usuarios, inicio de sesión, cierre de sesión y protección de rutas privadas.

Para cumplir con los requerimientos establecidos, el sistema fue implementado mediante dos tecnologías del lado del servidor: PHP y Java con Spring Boot. La primera versión utiliza PHP 8 y PDO para ejecutar operaciones sobre la base de datos. La documentación oficial de PHP recomienda utilizar consultas preparadas y parámetros para evitar la inclusión directa de datos ingresados por el usuario dentro de una sentencia SQL [1].

La segunda versión fue desarrollada con Java y Spring Boot. En esta implementación, Spring Security se encarga de administrar la autenticación, la autorización, el cierre de sesión y diferentes mecanismos de protección frente a ataques comunes [4]. El acceso a los datos se realiza mediante Spring Data JPA y repositorios, los cuales proporcionan métodos para desarrollar operaciones CRUD sobre las entidades del sistema [8].

También se aplicaron controles de seguridad recomendados por OWASP. El OWASP Top 10 reúne algunos de los riesgos más importantes para las aplicaciones web, entre los que se encuentran las fallas de control de acceso, los errores criptográficos y las fallas de identificación y autenticación [10]. En SGROAS se implementó prevención de Cross-Site Scripting, protección CSRF, almacenamiento seguro de contraseñas, consultas preparadas y cabeceras HTTP de seguridad.

Finalmente, se realiza una comparación técnica entre PHP y Java con Spring Boot, considerando criterios como seguridad por defecto, rendimiento, ecosistema, facilidad de aprendizaje, mantenimiento, escalabilidad, pruebas y disponibilidad de alojamiento. Esta comparación permite reconocer las ventajas y limitaciones de cada alternativa dentro del desarrollo de aplicaciones web dinámicas y seguras.

2. Desarrollo

2.1. Fundamento teórico

La programación web del lado del servidor constituye uno de los pilares fundamentales del desarrollo de aplicaciones modernas. En el servidor se procesa la lógica de negocio, se gestionan las conexiones con bases de datos, se controla la autenticación de usuarios y se generan las respuestas dinámicas que posteriormente son enviadas al navegador. En esta sección se estudian las principales tecnologías y plataformas empleadas para implementar esta capa de procesamiento, considerando sus características, ventajas, limitaciones y contextos de aplicación.

2.1.1. Tema 1: Introducción a la programación del lado del servidor

Este tema presenta los conceptos fundamentales relacionados con el funcionamiento de las aplicaciones web del lado del servidor. También se estudian PERL y PHP como tecnologías históricamente relevantes, y se establecen criterios generales para seleccionar una plataforma de desarrollo.

Subtema 1: Programación del lado del servidor: conceptos generales

La programación del lado del servidor, conocida en inglés como *server-side programming*, comprende el conjunto de técnicas, lenguajes y herramientas que permiten ejecutar código dentro de un servidor web. Este modelo se diferencia de la programación del lado del cliente, debido a que esta última se ejecuta directamente en el navegador del usuario.

Cuando una persona solicita una página o un recurso web, el navegador envía una petición mediante el protocolo HTTP. El servidor recibe la solicitud, identifica el recurso requerido, ejecuta la lógica correspondiente y genera una respuesta. Durante este proceso pueden realizarse operaciones como consultas a bases de datos, validación de formularios, autenticación de usuarios, cálculos, almacenamiento de información o comunicación con servicios externos. Finalmente, el servidor devuelve una respuesta en formatos como HTML, JSON, XML o archivos binarios.

Entre las principales características de la programación del lado del servidor se encuentran:

- **Procesamiento centralizado:** la lógica de negocio se ejecuta en un entorno administrado por el desarrollador o la organización, facilitando el mantenimiento, la actualización y la auditoría del código.
- **Acceso controlado a recursos sensibles:** el servidor puede conectarse con bases de datos, sistemas de archivos y servicios externos sin exponer directamente las credenciales ni la lógica interna al usuario.

- **Generación dinámica de contenido:** la respuesta puede adaptarse según el usuario autenticado, sus permisos, sus preferencias o los datos almacenados.
- **Independencia del cliente:** el código principal se ejecuta en el servidor y no depende completamente del navegador o dispositivo utilizado.

El funcionamiento habitual sigue el modelo petición–respuesta. El cliente envía una solicitud mediante métodos como GET, POST, PUT o DELETE. Un servidor web, como Apache, Nginx o IIS, recibe la petición y la entrega al motor de ejecución correspondiente. Después de procesar la lógica y comunicarse con los recursos necesarios, el servidor construye una respuesta y la devuelve al cliente.

Este ciclo se ejecuta constantemente durante la interacción con una aplicación web, por lo que el código del servidor debe ser eficiente, seguro, mantenible y escalable. Entre las tecnologías disponibles se encuentran PERL, PHP, Java con Servlets y JSP, C# con ASP.NET, JavaScript con Node.js y Python con frameworks como Django o Flask. La selección depende de factores como el rendimiento, la escalabilidad, el costo, el ecosistema y la disponibilidad de desarrolladores capacitados (Murach & Harris, 2022).

Subtema 2: PERL para aplicaciones web

PERL, cuyo nombre proviene de *Practical Extraction and Report Language*, es un lenguaje interpretado creado por Larry Wall en 1987. Durante la década de 1990 adquirió importancia en el desarrollo web debido a su capacidad para procesar texto, manipular cadenas y trabajar con expresiones regulares.

PERL se integró ampliamente con CGI, denominado *Common Gateway Interface*. CGI permitía que el servidor web ejecutara un programa externo para procesar una solicitud y generar contenido dinámico. Este mecanismo fue utilizado en numerosas aplicaciones web antes de la aparición y consolidación de plataformas más modernas.

Entre las características de PERL aplicadas al desarrollo web se encuentran:

- Manejo avanzado de expresiones regulares para procesar, validar y transformar cadenas de texto.
- Compatibilidad con CGI para generar páginas dinámicas a partir de scripts ejecutados por el servidor.
- Disponibilidad de módulos mediante CPAN, denominado *Comprehensive Perl Archive Network*.
- Capacidad para acceder a bases de datos, procesar archivos, enviar correos electrónicos y automatizar tareas.
- Sintaxis flexible y concisa, aunque puede dificultar la legibilidad cuando no se siguen reglas claras de programación.

Aunque PERL tuvo una participación importante durante las primeras etapas de la web dinámica, actualmente ha perdido protagonismo frente a tecnologías como PHP, Python, Java y JavaScript. Esto se relaciona con una sintaxis menos intuitiva para principiantes, una menor presencia de frameworks modernos y una reducción de su comunidad orientada al desarrollo web.

Sin embargo, PERL continúa siendo utilizado en administración de sistemas, procesamiento de registros, automatización, análisis de texto y mantenimiento de aplicaciones heredadas que todavía funcionan en determinadas organizaciones (Wall et al., 2000; Perl 5 Porters, s. f.).

Subtema 3: PHP: Hypertext Preprocessor

PHP es un lenguaje de propósito general especialmente orientado al desarrollo web del lado del servidor. Su nombre corresponde al acrónimo recursivo *PHP: Hypertext Preprocessor*. Fue creado inicialmente por Rasmus Lerdorf en 1994 y posteriormente evolucionó hasta convertirse en una de las tecnologías más utilizadas para desarrollar sitios y aplicaciones web dinámicas.

PHP es empleado por gestores de contenido como WordPress, Joomla y Drupal. También puede utilizarse directamente o mediante frameworks como Laravel, Symfony y CodeIgniter.

Entre sus principales características se encuentran:

- **Integración con HTML:** permite incorporar instrucciones PHP dentro de documentos HTML mediante etiquetas como `<?php ... ?>`.
- **Acceso a bases de datos:** ofrece soporte para MySQL, MariaDB, PostgreSQL y SQLite, entre otros sistemas.
- **Uso de PDO:** PHP Data Objects proporciona una interfaz común para trabajar con diferentes motores de bases de datos y utilizar consultas preparadas.
- **Curva de aprendizaje accesible:** su sintaxis comparte características con lenguajes como C y Java.
- **Ecosistema de frameworks:** dispone de herramientas que facilitan la aplicación del patrón Modelo–Vista–Controlador, la gestión de rutas y la inyección de dependencias.
- **Disponibilidad de alojamiento:** numerosos proveedores de hosting incluyen PHP, Apache y bases de datos compatibles.

El funcionamiento básico comienza cuando el servidor web recibe una petición dirigida a un archivo PHP. El archivo es entregado al intérprete del lenguaje, que ejecuta

las instrucciones, procesa las operaciones necesarias y genera una respuesta. Normalmente, el resultado es enviado al navegador como un documento HTML.

Las versiones PHP 8.x incorporan propiedades tipadas, funciones flecha, mejoras en el tratamiento de errores y soporte para compilación JIT. Estas características fortalecen el desarrollo de aplicaciones modernas y permiten aplicar una programación más estructurada (The PHP Group, s. f.; Murach & Harris, 2022).

Subtema 4: Comparativa tecnológica y criterios de selección

La selección de una tecnología del lado del servidor debe realizarse mediante criterios técnicos, económicos y organizacionales. No existe una plataforma adecuada para todos los proyectos, debido a que cada alternativa responde a necesidades diferentes.

Cuadro 1: Comparación general de tecnologías del lado del servidor

Criterio	PERL	PHP	Java / ASP.NET
Curva de aprendizaje	Media o alta debido a su sintaxis flexible y compleja.	Baja o media, con una sintaxis accesible y cercana al desarrollo web.	Alta, debido al uso de orientación a objetos, arquitecturas y frameworks.
Rendimiento	Adecuado para procesamiento de texto y automatización.	Adecuado para aplicaciones web y mejorado mediante OPcache y versiones recientes.	Alto, debido a la compilación a bytecode en Java o lenguaje intermedio en .NET.
Escalabilidad	Limitada en aplicaciones web modernas de gran tamaño.	Buena cuando se utilizan frameworks y arquitecturas organizadas.	Alta y orientada a sistemas empresariales.
Comunidad y soporte	Comunidad más reducida y menor presencia en nuevos desarrollos web.	Comunidad amplia y gran disponibilidad de documentación.	Amplio ecosistema respaldado por organizaciones como Oracle, Spring y Microsoft.
Costo de implementación	Bajo, debido a su naturaleza de software libre.	Bajo y ampliamente disponible en servicios de alojamiento.	Variable según la infraestructura; Java y .NET son multiplataforma, pero ciertos entornos empresariales pueden generar costos adicionales.

Para proyectos pequeños o medianos, con presupuesto limitado y tiempos de entrega reducidos, PHP suele ser una alternativa conveniente debido a su disponibilidad y facilidad de implementación (The PHP Group, s. f.; Murach & Harris, 2022).

PERL continúa siendo útil para automatización, tratamiento de texto y mantenimiento de sistemas heredados (Wall et al., 2000). En cambio, Java y ASP.NET son apropiados para aplicaciones empresariales que requieren tipado fuerte, escalabilidad, dis-

ponibilidad y arquitecturas organizadas (Oracle, s. f.; Microsoft, s. f.-a).

2.1.2. Tema 2: Desarrollo web con Java y JSP

Java constituye una plataforma ampliamente utilizada para desarrollar aplicaciones empresariales. Su portabilidad, tipado fuerte y orientación a objetos permiten crear sistemas mantenibles y escalables. En el desarrollo web se emplean tecnologías como Servlets, JSP, Jakarta EE y Spring Boot.

Subtema 1: Java para el desarrollo web: plataforma y herramientas

Java es un lenguaje orientado a objetos, multiplataforma y fuertemente tipado. Fue desarrollado inicialmente por Sun Microsystems y posteriormente pasó a formar parte del ecosistema de Oracle.

Para el desarrollo de aplicaciones empresariales, Java se apoyó históricamente en Java EE, actualmente denominado Jakarta EE. Esta plataforma define especificaciones para desarrollar aplicaciones web distribuidas, seguras y escalables.

Entre los principales componentes del ecosistema se encuentran:

- **Máquina Virtual de Java:** la JVM ejecuta el bytecode generado durante la compilación, permitiendo que una misma aplicación funcione en diferentes sistemas operativos.
- **Contenedores de Servlets:** herramientas como Apache Tomcat, Jetty, WildFly y GlassFish administran las solicitudes HTTP y el ciclo de vida de los componentes web.
- **Gestión de dependencias:** Maven y Gradle automatizan la compilación, la descarga de librerías y el empaquetado en archivos JAR o WAR.
- **Entornos de desarrollo:** IntelliJ IDEA, Eclipse y NetBeans proporcionan funciones de escritura, depuración y ejecución de proyectos Java.

Estas herramientas han permitido que Java sea utilizado en entornos corporativos, financieros, administrativos y bancarios, donde la estabilidad, la seguridad y el mantenimiento a largo plazo son prioritarios (Oracle, s. f.).

Subtema 2: Java Servlets

Los Servlets son componentes fundamentales del desarrollo web tradicional con Java. Un Servlet es una clase que procesa solicitudes HTTP y genera respuestas dinámicas. Se ejecuta dentro de un contenedor, como Apache Tomcat, que administra sus instancias y recursos.

A diferencia de CGI, donde podía crearse un nuevo proceso por cada solicitud, el contenedor mantiene los Servlets en memoria y administra múltiples peticiones. Esta característica mejora el rendimiento y reduce la creación repetida de procesos.

El ciclo de vida de un Servlet incluye tres métodos principales:

- **init()**: se ejecuta cuando el contenedor inicializa el Servlet y permite preparar recursos.
- **service()**: se ejecuta al recibir una solicitud. En clases derivadas de `HttpServlet`, normalmente se utilizan métodos como `doGet()` y `doPost()`.
- **destroy()**: se ejecuta cuando el componente es retirado de memoria y permite liberar recursos.

Los Servlets pueden recibir parámetros de formularios, validar información, acceder a bases de datos mediante JDBC, gestionar sesiones mediante `HttpSession` y generar respuestas HTML, JSON o XML.

Aunque ofrecen control sobre las solicitudes, generar documentos HTML directamente desde clases Java puede dificultar el mantenimiento. Esta necesidad impulsó la utilización de JSP como tecnología orientada a la presentación (Eclipse Foundation, s. f.-b).

Subtema 3: JavaServer Pages

JavaServer Pages permite combinar marcado HTML con contenido dinámico procesado mediante Java. En lugar de generar todo el HTML desde un Servlet mediante instrucciones de escritura, JSP permite redactar la estructura visual de la página e incorporar expresiones dinámicas.

Entre las construcciones tradicionales se encuentran:

- Scriptlets: `<% ... %>`.
- Expresiones: `<%= ... %>`.
- Directivas: `<%@ ... %>`.

Internamente, una página JSP es traducida y compilada por el contenedor como un Servlet. Por esta razón, JSP funciona como una capa de abstracción para simplificar la generación de contenido dinámico (Eclipse Foundation, s. f.-a).

Entre sus elementos se encuentran:

- **Directivas**: permiten importar paquetes, establecer configuraciones o incluir archivos.
- **JSTL**: proporciona etiquetas para condicionales, recorridos, formateo y otras operaciones comunes.
- **Expression Language**: permite acceder a datos mediante expresiones como `${usuario.nombre}`.

- **Objetos implícitos:** incluye elementos como `request`, `response`, `session` y `out`.

La combinación de Servlets y JSP dio lugar a una arquitectura semejante al patrón Modelo–Vista–Controlador. Los Servlets procesan las solicitudes y funcionan como controladores, mientras que JSP se utiliza como capa de presentación.

Subtema 4: Introducción a Spring Boot para desarrollo web

Spring Boot es un framework del ecosistema Spring que simplifica la creación de aplicaciones Java. Reduce la configuración manual y adopta el principio de convención sobre configuración.

Entre sus principales características se encuentran:

- **Servidor embebido:** una aplicación puede incluir Apache Tomcat, Jetty o Undertow sin requerir una instalación externa.
- **Inyección de dependencias:** el contenedor de Inversión de Control administra los componentes y sus relaciones.
- **Spring MVC:** permite definir controladores y rutas mediante anotaciones como `@Controller`, `@RestController`, `@GetMapping` y `@PostMapping`.
- **Spring Data JPA:** facilita el acceso a bases de datos mediante entidades y repositorios.
- **Starters:** dependencias como `spring-boot-starter-web` y `spring-boot-starter-data-jpa` agrupan las librerías necesarias para diferentes funcionalidades.
- **Autoconfiguración:** Spring Boot configura componentes automáticamente según las dependencias incorporadas.

Estas características permiten desarrollar aplicaciones web y servicios REST con una estructura basada en controladores, servicios, repositorios y entidades. Spring Boot ha reducido significativamente la necesidad de configurar manualmente Servlets y contenedores en proyectos nuevos (Spring, s. f.).

2.1.3. Tema 3: Desarrollo web con ASP.NET

ASP.NET es la plataforma de Microsoft destinada al desarrollo de aplicaciones web. Se encuentra integrada con .NET y utiliza principalmente el lenguaje C#. Su evolución hacia ASP.NET Core permitió incorporar compatibilidad multiplataforma, una arquitectura modular y mejores mecanismos para la configuración de aplicaciones.

Subtema 1: La plataforma .NET y ASP.NET

.NET es una plataforma de desarrollo que incluye un entorno de ejecución, bibliotecas y soporte para diferentes lenguajes. Entre sus componentes se encuentran el CLR, denominado *Common Language Runtime*, y la biblioteca de clases BCL.

ASP.NET es el framework de .NET orientado al desarrollo web. Es necesario diferenciar entre ASP.NET Framework y ASP.NET Core. La primera alternativa se encuentra vinculada principalmente con Windows e IIS, mientras que ASP.NET Core es de código abierto y puede ejecutarse en Windows, Linux y macOS.

Entre sus principales características se encuentran:

- Tipado fuerte y orientación a objetos mediante C#.
- Integración con Visual Studio y Visual Studio Code.
- Herramientas de depuración, autocompletado y refactorización.
- Acceso a bases de datos mediante Entity Framework Core.
- Compatibilidad con aplicaciones web, servicios REST y arquitecturas empresariales.
- Rendimiento competitivo mediante ASP.NET Core.

La plataforma facilita la detección temprana de errores y proporciona una estructura apropiada para aplicaciones de gran tamaño (Microsoft, s. f.-a).

Subtema 2: Estructura de un proyecto ASP.NET Core

ASP.NET Core utiliza una arquitectura modular. Un proyecto web puede incluir los siguientes elementos:

- **Program.cs**: constituye el punto de entrada. En este archivo se configura el servidor, se registran los servicios y se establece la canalización de middleware.
- **appsettings.json**: almacena configuraciones de la aplicación, cadenas de conexión y parámetros dependientes del entorno.
- **wwwroot**: contiene archivos estáticos como imágenes, hojas de estilo y scripts.
- **Pages**: contiene los archivos de Razor Pages cuando se utiliza el modelo orientado a páginas.
- **Controllers y Views**: almacenan controladores y vistas cuando se utiliza MVC.
- **Middleware**: corresponde a los componentes que procesan cada solicitud de manera secuencial.

El middleware puede encargarse de autenticación, enrutamiento, manejo de errores, compresión y registro de actividad.

ASP.NET Core incorpora el servidor Kestrel. En entornos de producción puede combinarse con Nginx o IIS como proxy inverso para administrar aspectos como certificados, balanceo de carga y conexiones externas (Microsoft, s. f.-b).

Subtema 3: Razor Pages y Razor Views

Razor es la sintaxis utilizada por ASP.NET Core para combinar HTML con expresiones de C#. Emplea el símbolo @ para incorporar contenido dinámico dentro de las páginas.

ASP.NET Core dispone de dos enfoques principales:

- **Razor Pages:** cada archivo `.cshtml` se encuentra asociado con una clase `PageModel`. En esta clase pueden definirse métodos como `OnGet()` y `OnPost()`.
- **Razor Views:** utiliza el patrón Modelo–Vista–Controlador. Los controladores procesan las solicitudes y retornan vistas ubicadas dentro de la carpeta `Views`.

Razor Pages resulta conveniente para formularios y aplicaciones centradas en páginas específicas (Microsoft, s. f.-d). El enfoque MVC proporciona una separación más explícita entre la presentación, la lógica de control y los datos.

Ambas alternativas permiten utilizar:

- *Layouts* para definir plantillas compartidas.
- *Partial Views* para reutilizar fragmentos de interfaz.
- *Tag Helpers* para generar elementos y atributos HTML desde el servidor.

Subtema 4: Formularios, validación y seguridad en ASP.NET

Los formularios representan uno de los principales puntos de entrada de información en una aplicación web. Por esta razón, ASP.NET Core incorpora mecanismos para recibir, validar y proteger los datos proporcionados por los usuarios.

El *Model Binding* asocia automáticamente los campos enviados con las propiedades de un objeto C#. Los *Tag Helpers*, como `asp-for`, generan nombres, valores y elementos de validación a partir de las propiedades del modelo.

Las reglas pueden definirse mediante *Data Annotations*, entre ellas:

- `[Required]` para campos obligatorios.
- `[StringLength]` y `[MaxLength]` para limitar caracteres.
- `[Range]` para definir intervalos numéricos.
- `[EmailAddress]` para validar correos electrónicos.
- `[RegularExpression]` para comprobar patrones.

Las validaciones son verificadas en el servidor mediante `ModelState.IsValid`. También pueden reflejarse en el cliente mediante atributos HTML y scripts de validación. Sin embargo, la comprobación del servidor debe mantenerse como mecanismo definitivo.

En materia de seguridad, ASP.NET Core proporciona:

- Tokens antifalsificación para reducir ataques CSRF.
- Codificación automática de salidas en Razor para disminuir ataques XSS.
- ASP.NET Core Identity para autenticación y autorización.
- Almacenamiento seguro de contraseñas mediante hash.
- Administración de roles y permisos.
- Integración con proveedores externos de autenticación.

Estas herramientas permiten construir formularios y sistemas de autenticación con medidas de seguridad incorporadas, especialmente útiles en aplicaciones empresariales (Microsoft, s. f.-c).

2.2. Descripción general del sistema

SGROAS es una aplicación web dinámica diseñada para gestionar información de conductores y controlar el acceso de los usuarios mediante un sistema de autenticación. La aplicación dispone de una interfaz principal desde la cual el usuario puede visualizar información resumida, consultar los conductores registrados y acceder a las diferentes operaciones disponibles.

El proceso comienza con el registro de una cuenta. En este formulario se solicita el nombre completo, el correo electrónico, la contraseña y su confirmación. Posteriormente, el usuario puede iniciar sesión utilizando las credenciales registradas. Cuando los datos son correctos, el sistema crea una sesión autenticada y permite el acceso a las rutas privadas.

Después de iniciar sesión, el usuario es dirigido al dashboard principal. Esta pantalla presenta la cantidad total de conductores registrados, el número de conductores activos, la cantidad de registros inactivos o suspendidos y el estado general del sistema. También incluye accesos directos para consultar los conductores o registrar uno nuevo.

El módulo principal permite listar los conductores almacenados en la base de datos. Cada registro muestra la cédula, nombres y apellidos, teléfono, tipo de licencia, número de licencia, fecha de vencimiento y estado. Desde esta interfaz se pueden realizar las operaciones de creación, consulta, actualización y desactivación de registros.

2.3. Tecnologías utilizadas

Para el desarrollo de la práctica se utilizaron dos tecnologías del lado del servidor. Ambas implementaciones mantienen las mismas funcionalidades principales, pero utilizan diferentes herramientas para gestionar la autenticación, la seguridad y el acceso a la base de datos.

Cuadro 2: Tecnologías utilizadas en el desarrollo de SGROAS

Elemento	Implementación PHP	Implementación Java
Lenguaje	PHP 8	Java 21
Tecnología principal	PHP con organización por capas	Spring Boot
Autenticación	Sesiones PHP	Spring Security
Acceso a datos	PDO y consultas preparadas	Spring Data JPA
Protección CSRF	Token generado y validado en la sesión	Protección integrada de Spring Security
Contraseñas	<code>password_hash()</code> y <code>password_verify()</code>	BCrypt mediante <code>PasswordEncoder</code>
Vistas	HTML, CSS y plantillas PHP	Thymeleaf
Servidor local	Apache	Servidor embebido de Spring Boot

La implementación en PHP utiliza PDO para establecer la comunicación con la base de datos. El método `PDO::prepare()` permite preparar una sentencia SQL con marcadores de parámetros, mientras que `execute()` incorpora posteriormente los valores correspondientes. Este procedimiento evita concatenar directamente los datos proporcionados por los usuarios y ayuda a prevenir ataques de inyección SQL [1].

En la versión desarrollada con Java se utiliza Spring Boot como tecnología principal. Spring Security administra la autenticación, la autorización, la protección CSRF y las cabeceras HTTP de seguridad [4]. Por otra parte, Spring Data JPA permite definir repositorios asociados con las entidades y proporciona métodos CRUD para consultar, registrar, actualizar y eliminar información [8].

2.4. Funcionalidades implementadas

El sistema SGROAS incorpora las siguientes funcionalidades:

1. Registro de nuevos usuarios mediante nombre completo, correo electrónico y contraseña.
2. Inicio de sesión mediante correo electrónico y contraseña.
3. Verificación de las credenciales antes de permitir el acceso al sistema.
4. Cierre de sesión y finalización de la sesión activa.
5. Protección de rutas privadas para impedir el ingreso de usuarios no autenticados.

6. Visualización de un dashboard con información resumida de los conductores.
7. Registro de nuevos conductores mediante un formulario web.
8. Consulta y búsqueda de conductores registrados.
9. Modificación de la información de un conductor.
10. Desactivación de conductores desde el listado principal.

El formulario de conductores solicita información como cédula, nombres, apellidos, teléfono, correo electrónico, estado, tipo de licencia, número de licencia y fecha de vencimiento. Los datos son procesados en el servidor antes de almacenarse en la base de datos.

2.5. Controles de seguridad implementados

La seguridad representa un elemento indispensable en el desarrollo de aplicaciones web. OWASP identifica riesgos relacionados con el control de acceso, el almacenamiento incorrecto de información sensible, la autenticación y la inyección de código [10]. Por esta razón, en ambas versiones de SGROAS se incorporaron diferentes controles de protección.

2.5.1. Prevención de Cross-Site Scripting

Cross-Site Scripting, conocido como XSS, es un ataque que permite introducir contenido malicioso en una aplicación para que posteriormente sea ejecutado en el navegador de otro usuario. Este problema puede ocurrir cuando la aplicación presenta datos sin aplicar una codificación adecuada a las salidas [11].

En la implementación PHP se utiliza la función:

```
1 <?= htmlspecialchars($valor, ENT_QUOTES, 'UTF-8'); ?>
```

Listing 1: Saneamiento de una salida en PHP

La función `htmlspecialchars()` convierte caracteres especiales en entidades HTML, evitando que una cadena ingresada por el usuario sea interpretada directamente como una etiqueta o un script.

En Spring Boot se utilizan atributos de Thymeleaf como `th:text` y `th:field`. Thymeleaf escapa de forma predeterminada el contenido presentado con `th:text`, mientras que los valores procesados por `th:field` también se generan con escape HTML [9].

2.5.2. Protección contra ataques CSRF

Cross-Site Request Forgery, conocido como CSRF, ocurre cuando un sitio externo provoca que el navegador de un usuario autenticado envíe una acción no deseada hacia

una aplicación legítima. El servidor podría interpretar dicha petición como válida porque incluye automáticamente la información de la sesión activa [12].

En PHP se genera un token aleatorio y se almacena en la sesión:

```
1      if (empty($_SESSION['csrf_token'])) {
2          $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
3      }
```

Listing 2: Generación de un token CSRF en PHP

El token se incluye posteriormente dentro del formulario:

```
1      <input
2      type="hidden"
3      name="csrf_token"
4      value="<?= htmlspecialchars(
5      $_SESSION['csrf_token'],
6      ENT_QUOTES,
7      'UTF-8'
8      ); ?>"
9      >
```

Listing 3: Token CSRF dentro de un formulario

Antes de ejecutar una operación, el valor recibido debe compararse con el valor almacenado en la sesión. En la implementación con Spring Boot, Spring Security mantiene habilitada por defecto la protección CSRF para métodos HTTP considerados inseguros, como las solicitudes POST [5].

2.5.3. Almacenamiento seguro de contraseñas

Las contraseñas no deben guardarse directamente en texto plano. En PHP se utiliza `password_hash()` para generar una representación segura de la contraseña. La información necesaria sobre el algoritmo y sus parámetros se almacena dentro del propio hash [2].

```
1      $passwordHash = password_hash(
2      $password,
3      PASSWORD_ARGON2ID
4      );
```

Listing 4: Generación del hash de una contraseña en PHP

Durante el inicio de sesión se utiliza `password_verify()` para comprobar si la contraseña ingresada corresponde con el hash almacenado [3].

```
1      if (password_verify($password, $passwordHash)) {
```

```

2         // Credenciales correctas
3     }

```

Listing 5: Verificación de una contraseña en PHP

En Spring Boot se utiliza la interfaz `PasswordEncoder`. Spring Security define esta interfaz para realizar una transformación unidireccional de la contraseña antes de almacenarla [6].

```

1     @Bean
2     public PasswordEncoder passwordEncoder() {
3         return new BCryptPasswordEncoder(12);
4     }

```

Listing 6: Codificación de una contraseña con BCrypt

2.5.4. Cabeceras HTTP de seguridad

Las cabeceras HTTP permiten establecer instrucciones de seguridad que deben ser interpretadas por el navegador. Entre las cabeceras aplicadas en el sistema se encuentran:

- Content-Security-Policy.
- X-Frame-Options.
- X-Content-Type-Options.
- Strict-Transport-Security.
- Referrer-Policy.

Estas cabeceras ayudan a limitar la carga de recursos, impedir la incorporación no autorizada de la aplicación dentro de marcos externos, evitar la interpretación incorrecta del tipo de contenido y reforzar el uso de conexiones seguras. Spring Security proporciona un conjunto predeterminado de cabeceras HTTP y permite personalizarlas mediante la configuración de seguridad [7].

2.5.5. Prevención de inyección SQL

La inyección SQL puede presentarse cuando una aplicación construye consultas mediante la concatenación directa de información proporcionada por el usuario. OWASP recomienda utilizar consultas parametrizadas o prepared statements como una de las principales defensas frente a esta vulnerabilidad [13].

En PHP se utilizan consultas preparadas con PDO:

```

1      $sql = 'SELECT * FROM usuarios WHERE email = :email';
2
3      $stmt = $pdo->prepare($sql);
4
5      $stmt->execute([
6          'email' => $email
7      ]);
8
9      $usuario = $stmt->fetch();

```

Listing 7: Consulta preparada mediante PDO

En la versión Spring Boot, el acceso a la información se realiza mediante repositorios de Spring Data JPA. Estos repositorios proporcionan operaciones CRUD y permiten trabajar con consultas derivadas o parametrizadas sin construir manualmente las instrucciones SQL principales [8].

3. Resultados obtenidos

El desarrollo de SGROAS permitió obtener dos aplicaciones web funcionales con características equivalentes, implementadas mediante PHP y Java con Spring Boot. En ambas versiones se incorporaron mecanismos de autenticación, protección de rutas, gestión de sesiones y un módulo CRUD destinado a la administración de conductores.

Las pruebas funcionales se realizaron en un entorno local. La implementación PHP fue ejecutada mediante Apache, mientras que la versión Java utilizó el servidor web embebido proporcionado por Spring Boot. A continuación, se presentan las principales interfaces obtenidas durante la ejecución de cada aplicación.

3.1. Resultados de la implementación en PHP

La versión desarrollada en PHP presenta inicialmente un formulario de autenticación que solicita el correo electrónico y la contraseña del usuario. El sistema comprueba las credenciales registradas antes de permitir el ingreso a las páginas protegidas.

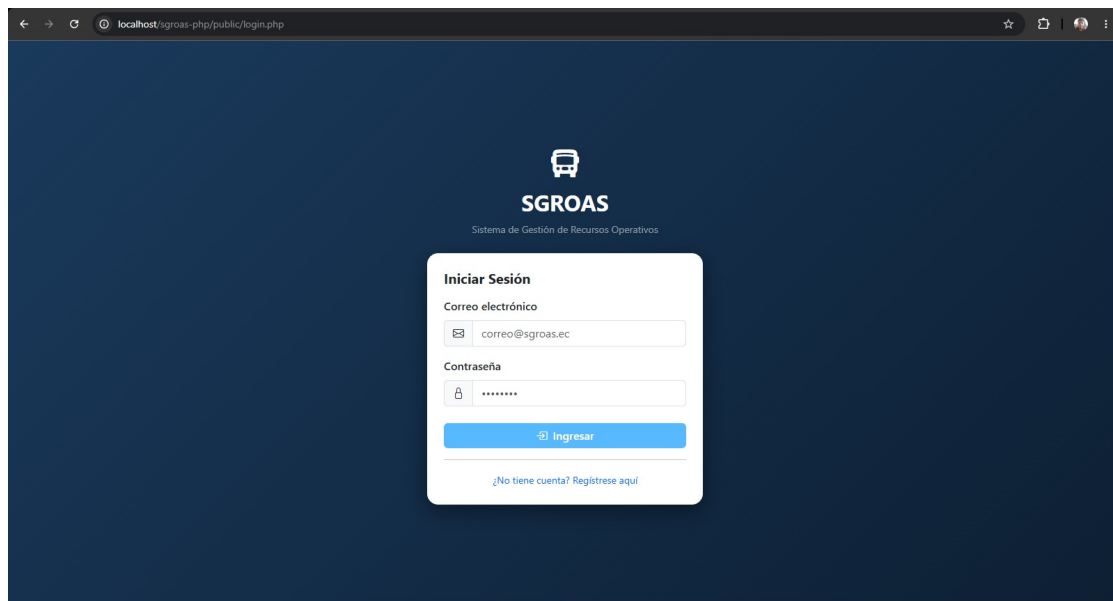


Figura 1: Pantalla de inicio de sesión de SGROAS en PHP.

Como se observa en la Figura 1, el formulario contiene los campos necesarios para ingresar las credenciales. También dispone de un enlace hacia la pantalla de creación de una nueva cuenta.

El registro de usuarios permite almacenar el nombre completo, correo electrónico y contraseña. El formulario solicita la confirmación de la contraseña y comunica los requisitos mínimos que debe cumplir.

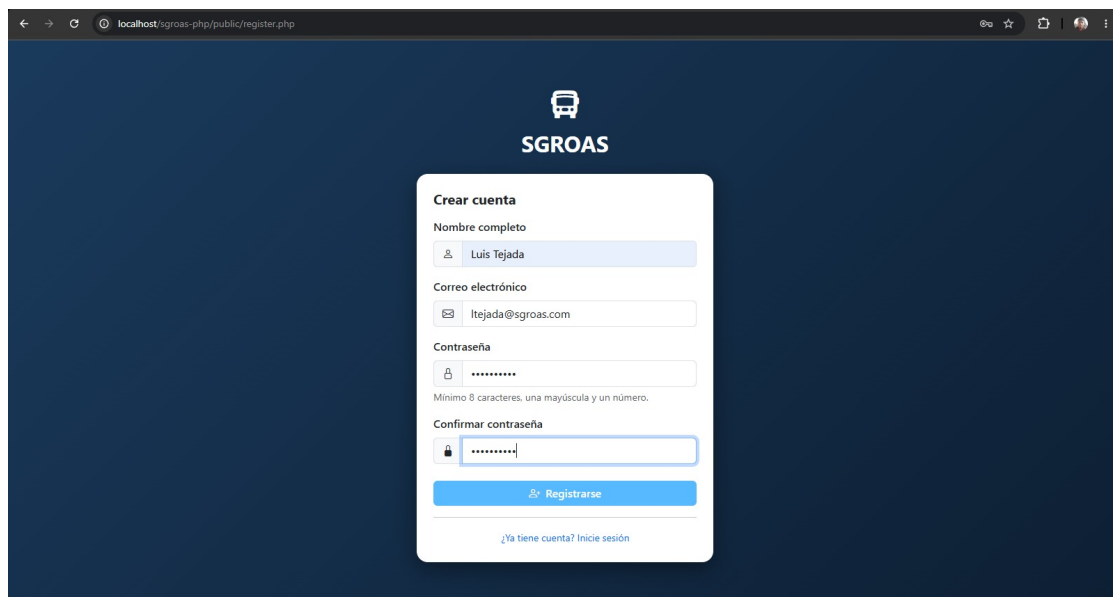


Figura 2: Formulario de creación de cuenta en PHP.

Las contraseñas procesadas mediante este formulario no son almacenadas directamente en texto plano. La implementación utiliza las funciones de hash incorporadas en PHP para proteger las credenciales antes de registrarlas en la base de datos.

Una vez autenticado, el usuario es dirigido al dashboard principal. Esta interfaz presenta un resumen del número de conductores registrados, activos e inactivos o suspendidos.

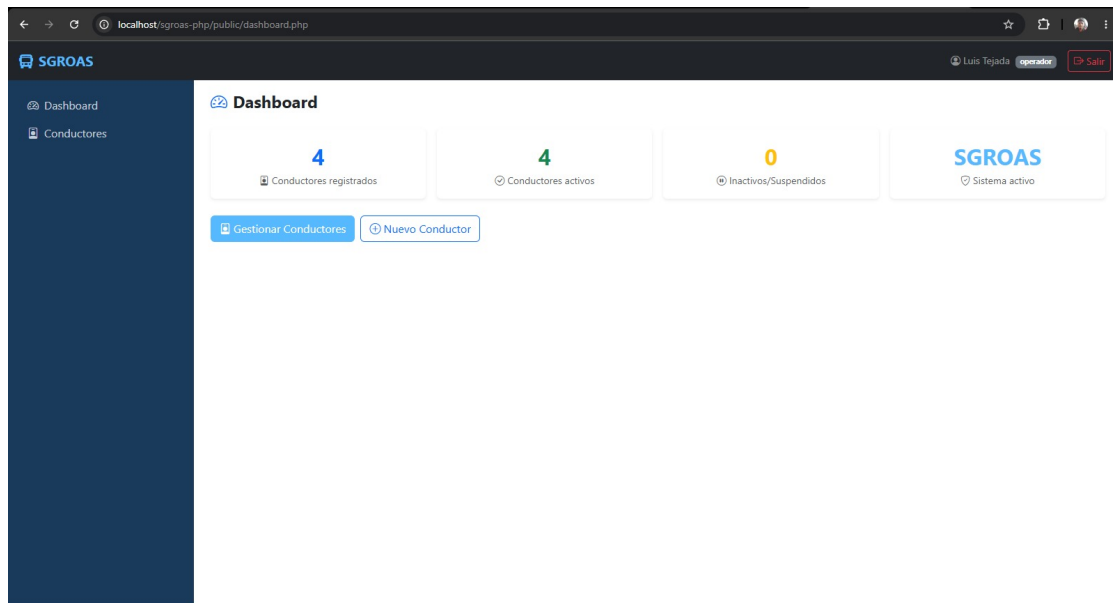


Figura 3: Dashboard principal de SGROAS en PHP.

En la Figura 3 se observa que el sistema reconoce al usuario autenticado y muestra su nombre y rol en la barra superior. También se encuentran disponibles los botones para acceder al listado de conductores o registrar un nuevo conductor.

La interfaz de conductores muestra los registros almacenados en forma de tabla. Cada fila presenta la cédula, nombre completo, teléfono, licencia, fecha de vencimiento, estado y acciones disponibles.

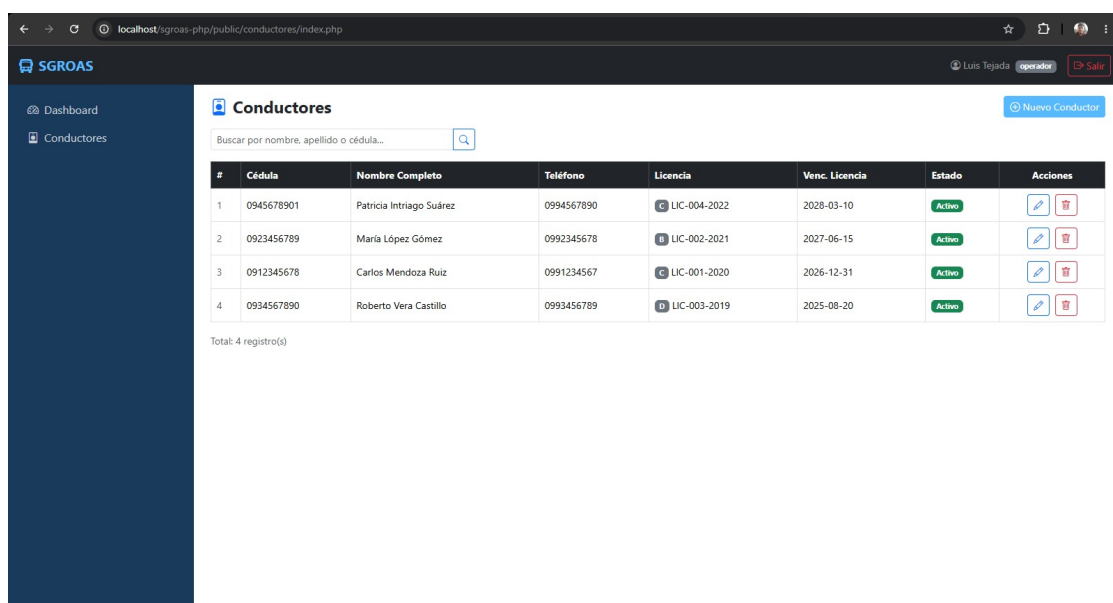
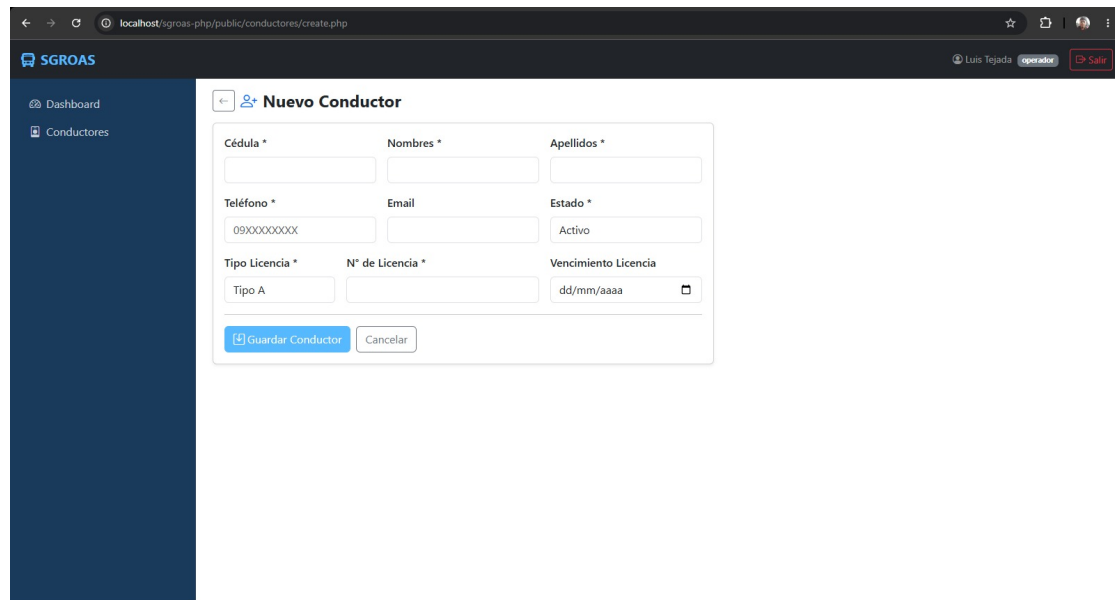


Figura 4: Listado y gestión de conductores en PHP.

El módulo dispone de un campo de búsqueda que permite localizar conductores por nombre, apellido o cédula. Además, se incluyen botones para editar y desactivar cada registro, completando las operaciones principales del módulo CRUD.

El registro de un conductor se realiza mediante un formulario que solicita sus datos personales y la información relacionada con su licencia.



The screenshot shows a web browser window with the URL `localhost/sgroas-php/public/conductores/create.php`. The page title is "Nuevo Conductor". On the left is a dark blue sidebar with the "SGROAS" logo and navigation links for "Dashboard" and "Conductores". The main content area contains a form with the following fields:

- Cédula ***: Text input field.
- Nombres ***: Text input field.
- Apellidos ***: Text input field.
- Teléfono ***: Text input field with a placeholder "09XXXXXXXX".
- Email**: Text input field.
- Estado ***: Dropdown menu with "Activo" selected.
- Tipo Licencia ***: Dropdown menu with "Tipo A" selected.
- N° de Licencia ***: Text input field.
- Vencimiento Licencia**: Text input field with a placeholder "dd/mm/aaaa" and a calendar icon.

At the bottom of the form are two buttons: "Guardar Conductor" (blue) and "Cancelar" (white).

Figura 5: Formulario para registrar un conductor en PHP.

El formulario mostrado en la Figura 5 solicita la cédula, nombres, apellidos, teléfono, correo electrónico, estado, tipo de licencia, número de licencia y fecha de vencimiento. Los campos obligatorios se encuentran identificados mediante un asterisco.

3.2. Resultados de la implementación en Java con Spring Boot

La segunda versión de SGROAS fue desarrollada utilizando Java y Spring Boot. Esta implementación conserva las funcionalidades principales de la versión PHP, pero utiliza Spring Security para administrar la autenticación, la protección de rutas y la gestión de sesiones [4].

La primera pantalla corresponde al formulario de inicio de sesión.

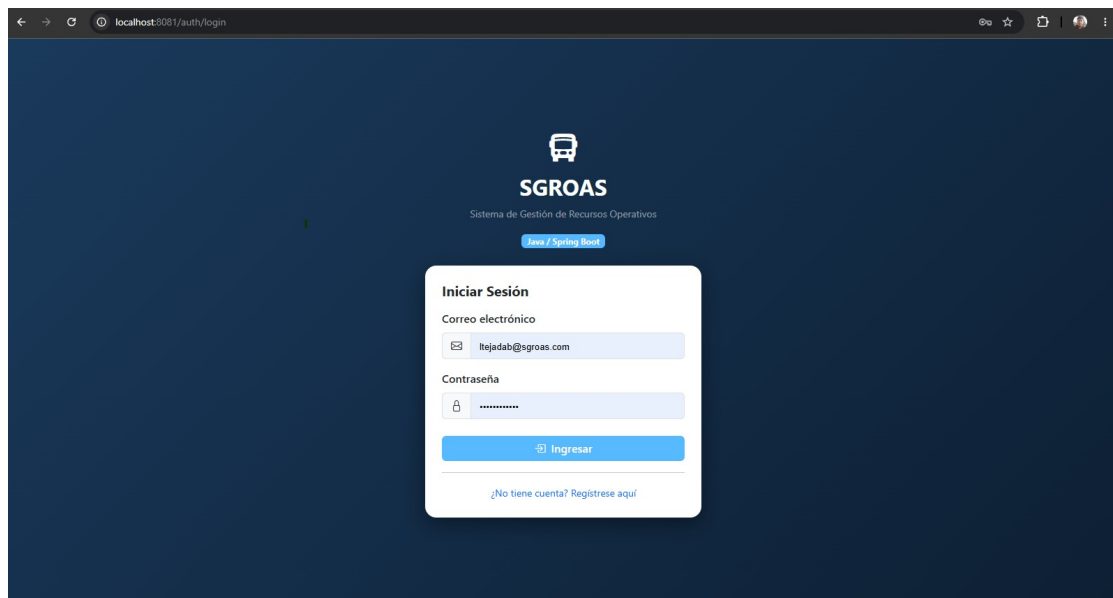


Figura 6: Pantalla de inicio de sesión en Java con Spring Boot.

La dirección mostrada en el navegador corresponde a la ruta de autenticación de la aplicación ejecutada en el puerto 8081. El formulario permite ingresar el correo electrónico y la contraseña del usuario.

La aplicación también dispone de una interfaz para registrar nuevas cuentas.

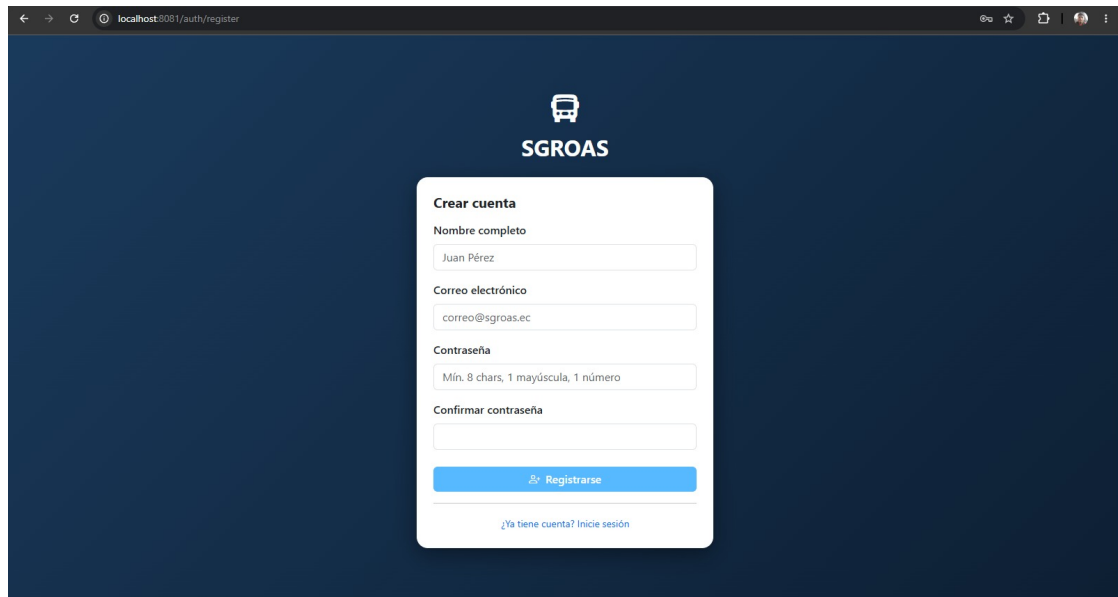


Figura 7: Formulario de creación de cuenta en Spring Boot.

El formulario incluye validaciones para el nombre, correo electrónico, contraseña y confirmación. La contraseña debe tener al menos ocho caracteres, una letra mayúscula y un número.

Después de completar correctamente el proceso de autenticación, Spring Security permite el acceso al dashboard protegido.

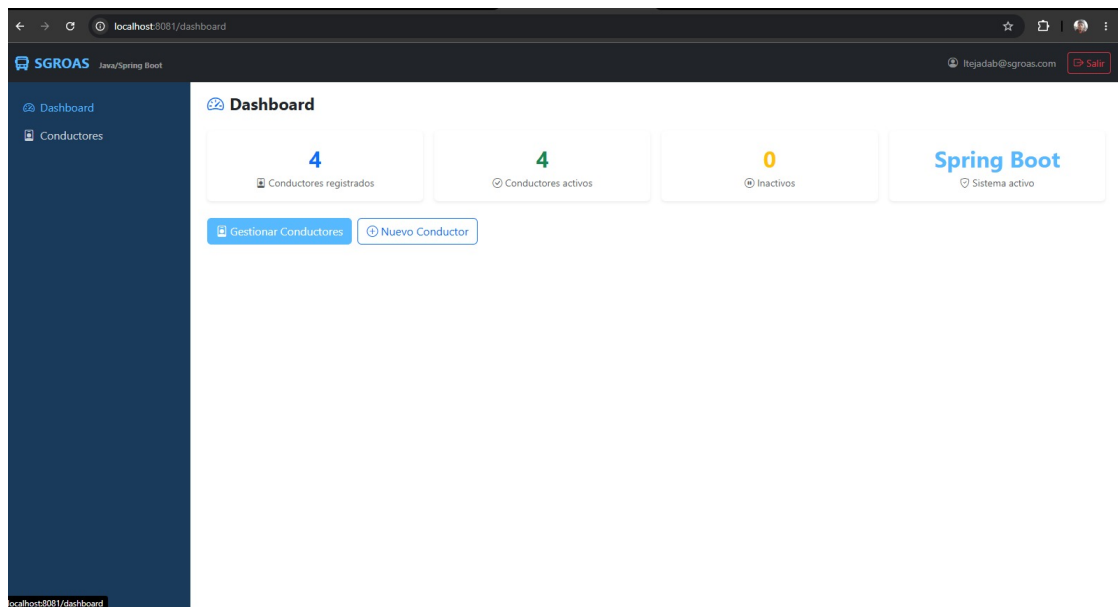
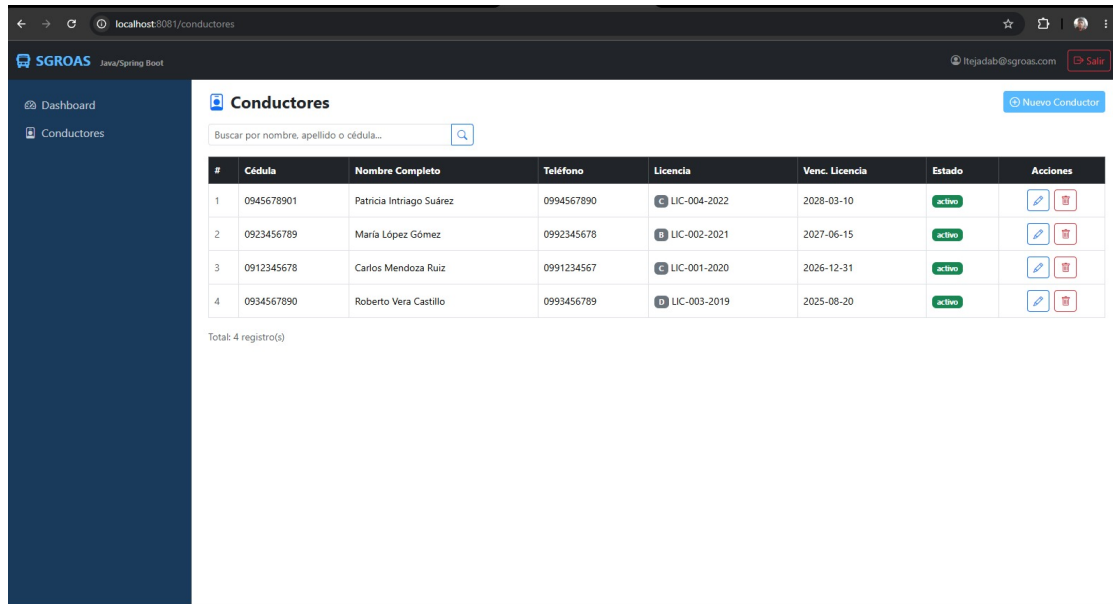


Figura 8: Dashboard principal desarrollado con Spring Boot.

La Figura 8 presenta cuatro conductores registrados y activos. También identifica a Spring Boot como la tecnología utilizada y muestra el correo electrónico del usuario autenticado.

La gestión de conductores se presenta mediante una tabla semejante a la implementada en PHP.



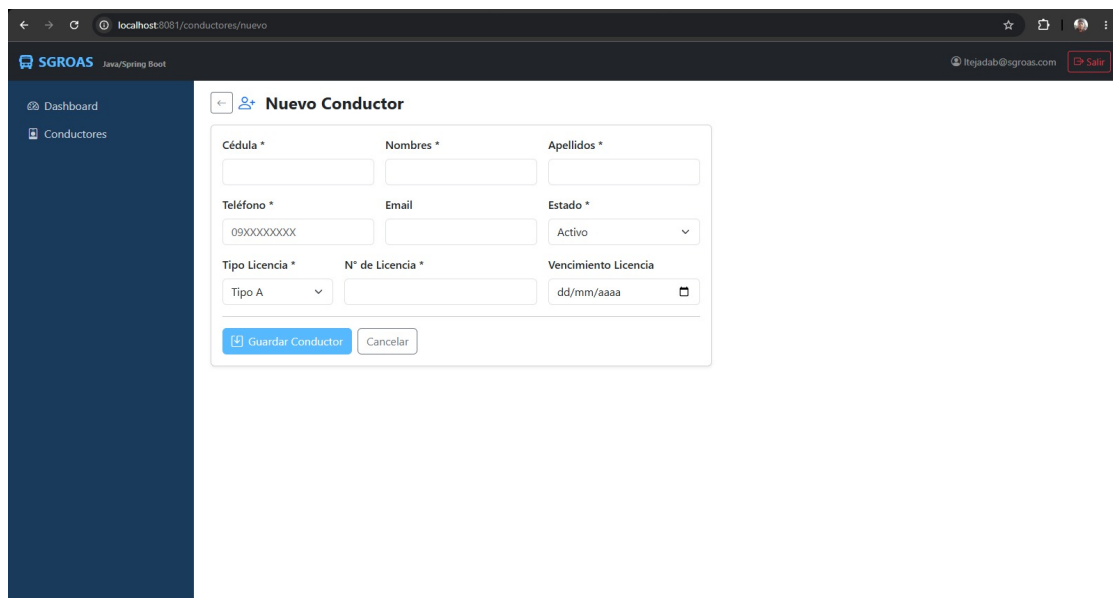
#	Cédula	Nombre Completo	Teléfono	Licencia	Venc. Licencia	Estado	Acciones
1	0945678901	Patricia Intriago Suárez	0994567890	LIC-004-2022	2028-03-10	activo	Editar Eliminar
2	0923456789	Maria López Gómez	0992345678	LIC-002-2021	2027-06-15	activo	Editar Eliminar
3	0912345678	Carlos Mendoza Ruiz	0991234567	LIC-001-2020	2026-12-31	activo	Editar Eliminar
4	0934567890	Roberto Vera Castillo	0993456789	LIC-003-2019	2025-08-20	activo	Editar Eliminar

Total: 4 registro(s)

Figura 9: Listado y gestión de conductores en Spring Boot.

Los registros pueden ser consultados, buscados, modificados y desactivados. Spring Data JPA facilita la implementación de estas operaciones mediante repositorios asociados con las entidades del sistema [8].

Finalmente, la aplicación dispone de un formulario para registrar nuevos conductores.



Nuevo Conductor

Cédula *

Nombres *

Apellidos *

Teléfono *

Email

Estado *

Tipo Licencia *

N° de Licencia *

Vencimiento Licencia

Guardar Conductor

Cancelar

Figura 10: Formulario para registrar un conductor en Spring Boot.

La información solicitada coincide con la versión PHP, lo cual permite comparar las dos tecnologías utilizando las mismas funcionalidades y condiciones de operación.

3.3. Resultado del análisis estático con PHPStan

Como parte de la validación del código fuente se utilizó PHPStan, una herramienta de análisis estático para PHP. La revisión se realizó sobre el archivo `Conductor.php`, correspondiente al modelo utilizado para representar los datos de los conductores.

El análisis fue configurado en el nivel 5, permitiendo revisar los tipos de datos, los parámetros de los métodos, los valores retornados y la consistencia general del código sin necesidad de ejecutar la aplicación.

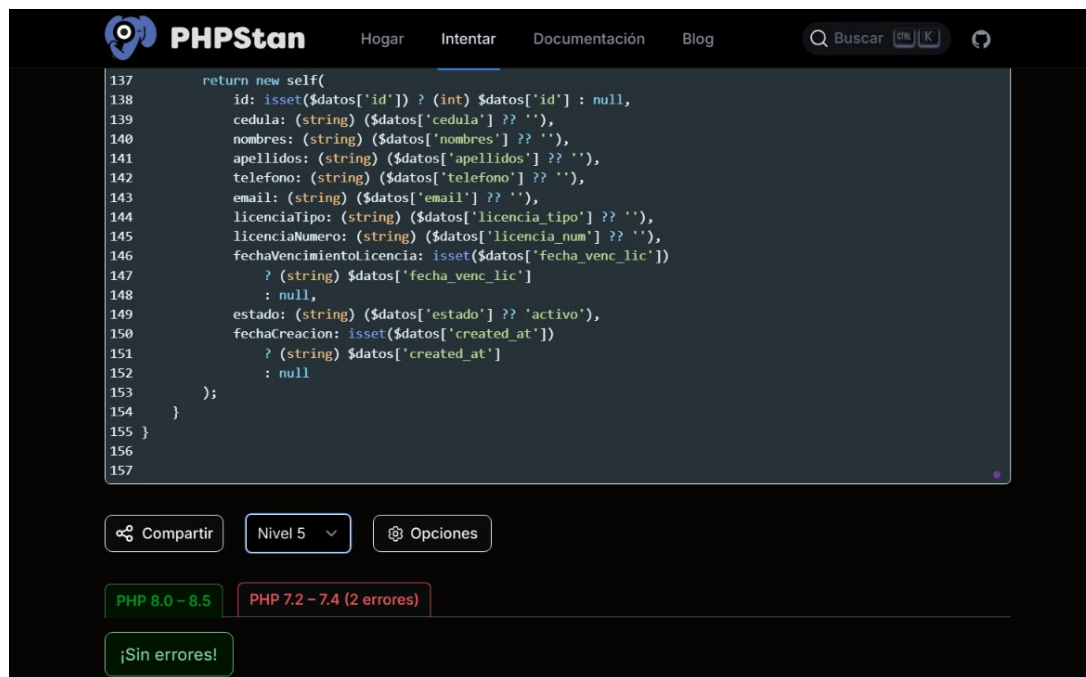


Figura 11: Resultado del análisis estático realizado con PHPStan.

La Figura 11 presenta el resultado generado por la herramienta. La revisión permitió comprobar que la estructura del modelo, las declaraciones de tipos y los métodos definidos mantienen una organización adecuada para el funcionamiento de la aplicación.

El análisis finalizó sin reportar errores en el archivo evaluado, lo que indica que el modelo cumple con las comprobaciones aplicadas por PHPStan en el nivel seleccionado.

4. Comparación entre PHP y Java con Spring Boot

Las dos tecnologías permitieron desarrollar las funcionalidades solicitadas. Sin embargo, presentan diferencias en su configuración, arquitectura, seguridad y facilidad de mantenimiento. PHP proporciona una implementación directa y flexible, mientras que Spring Boot incorpora una estructura más amplia basada en controladores, servicios, repositorios e inyección de dependencias.

La Tabla 3 presenta la comparación de ambas alternativas mediante diez criterios técnicos.

Cuadro 3: Comparación técnica entre PHP y Java con Spring Boot.

Criterio	PHP	Java con Spring Boot
Cantidad de código	Requiere menos configuración inicial y permite implementar rápidamente formularios, sesiones y consultas.	Requiere más clases y configuración inicial, debido a su arquitectura basada en controladores, servicios, entidades y repositorios.
Curva de aprendizaje	Su sintaxis y ejecución directa facilitan el aprendizaje inicial de aplicaciones web dinámicas.	Presenta una curva de aprendizaje mayor por el uso de orientación a objetos, anotaciones, inyección de dependencias y configuración de seguridad.
Seguridad por defecto	Varias medidas deben configurarse manualmente, como tokens CSRF, cabeceras HTTP, sesiones y saneamiento de salidas.	Spring Security proporciona mecanismos integrados para autenticación, autorización, CSRF, cierre de sesión y cabeceras HTTP [4].
Acceso a datos	PDO permite ejecutar consultas preparadas y controlar directamente las sentencias SQL [1].	Spring Data JPA abstrae gran parte del acceso a datos mediante entidades y repositorios [8].
Rendimiento	Es una alternativa ligera y adecuada para aplicaciones pequeñas y medianas con solicitudes web convencionales.	La máquina virtual de Java requiere más recursos iniciales, pero proporciona herramientas apropiadas para aplicaciones empresariales y servicios de mayor tamaño.

Cuadro 3 – Continuación

Criterio	PHP	Java con Spring Boot
Ecosistema	Dispone de Composer y Packagist para administrar paquetes y dependencias del proyecto.	Utiliza Maven o Gradle y cuenta con un amplio ecosistema de proyectos pertenecientes a Spring.
Alojamiento	Puede ejecutarse fácilmente en servidores que incorporen PHP y Apache o Nginx.	Requiere un entorno compatible con Java o el despliegue de la aplicación mediante un contenedor o servicio apropiado.
Pruebas	Puede utilizar PHPUnit, aunque su incorporación depende de la estructura definida por el equipo.	Dispone de JUnit, Spring Test y herramientas integradas para probar controladores, servicios y repositorios.
Mantenimiento y escalabilidad	Su flexibilidad permite desarrollos rápidos, pero requiere establecer reglas claras de organización para proyectos grandes.	Su estructura por capas facilita la separación de responsabilidades, el mantenimiento y la ampliación de sistemas grandes.
Aplicación recomendada	Resulta conveniente para aplicaciones pequeñas y medianas, sistemas administrativos y proyectos con recursos limitados.	Resulta conveniente para plataformas empresariales, servicios escalables y proyectos que requieren una arquitectura fuertemente organizada.

A partir de la comparación se observa que PHP permitió construir la aplicación con una configuración más directa y una menor cantidad de componentes. No obstante, varias medidas de seguridad tuvieron que ser implementadas manualmente.

Spring Boot requirió una estructura inicial más amplia, pero proporcionó mecanismos integrados que facilitaron la protección de rutas, la autenticación, el acceso a datos y la separación de responsabilidades. Por esta razón, su utilización puede resultar especialmente conveniente en proyectos grandes o que requieran crecimiento continuo.

Para SGROAS, ambas tecnologías cumplieron los requisitos establecidos. PHP destacó por su simplicidad y rapidez de implementación, mientras que Spring Boot sobresalió por su arquitectura organizada y su seguridad incorporada.

5. Análisis de la práctica

5.1. Tecnología con mayor seguridad por defecto

A partir de las implementaciones realizadas, Java con Spring Boot presentó una mayor cantidad de mecanismos de seguridad incorporados por defecto. Spring Security administra la autenticación, la autorización, la protección de rutas, los tokens CSRF, el cierre de sesión y diferentes cabeceras HTTP mediante una configuración centralizada [4]. Esto disminuye la necesidad de programar manualmente cada medida de protección.

En PHP también fue posible implementar todos los controles solicitados. Sin embargo, fue necesario desarrollar directamente la generación y validación de tokens CSRF, la gestión de sesiones, la protección de rutas y la configuración de cabeceras HTTP. Por esta razón, la seguridad de una aplicación PHP depende en mayor medida de las decisiones tomadas por el desarrollador.

Spring Boot puede considerarse la alternativa más segura por defecto dentro de esta práctica. No obstante, PHP también alcanza un nivel adecuado de protección cuando se aplican correctamente las recomendaciones de OWASP, las consultas preparadas, el saneamiento de salidas y el almacenamiento seguro de contraseñas.

5.2. Tecnología recomendada para el contexto del proyecto

Para proyectos pequeños o medianos que necesiten una implementación rápida, mantenimiento sencillo y una menor cantidad de configuración inicial, PHP representa una alternativa conveniente. Su ejecución mediante servidores como Apache permite desarrollar aplicaciones administrativas sin requerir una arquitectura demasiado compleja.

En cambio, Java con Spring Boot resulta apropiado para sistemas que necesitan una estructura por capas, crecimiento continuo, pruebas automatizadas, integración con múltiples servicios y una administración más centralizada de la seguridad. Su arquitectura facilita la separación entre controladores, servicios, repositorios y entidades.

Considerando las características de SGROAS, PHP puede utilizarse cuando el sistema tenga un alcance institucional limitado y una infraestructura sencilla. Spring Boot sería recomendable si la aplicación debe crecer, incorporar nuevos módulos, integrarse con otros servicios o atender una mayor cantidad de usuarios. La decisión final debe considerar el tamaño del proyecto, los conocimientos del equipo y los recursos disponibles.

5.3. Resultado obtenido mediante PHPStan

El análisis estático se realizó sobre el archivo `Conductor.php`, correspondiente al modelo empleado para representar la información de los conductores. Para la validación se utilizó PHPStan en el nivel 5.

La herramienta revisó las declaraciones de tipos, los parámetros recibidos por los métodos, los valores retornados y la estructura general de la clase. El proceso finalizó sin reportar errores, lo cual indica que el archivo evaluado cumple con las comprobaciones aplicadas en el nivel seleccionado.

Este resultado no reemplaza las pruebas funcionales de la aplicación, debido a que PHPStan analiza el código sin ejecutar el sistema. Sin embargo, permite detectar anticipadamente inconsistencias relacionadas con tipos de datos, métodos y variables, contribuyendo a mejorar la calidad y el mantenimiento del código fuente.

5.4. Integración con el frontend desarrollado en la Unidad 1

El backend de SGROAS puede integrarse con el frontend desarrollado previamente mediante solicitudes HTTP. Los formularios de registro e inicio de sesión envían los datos al servidor utilizando el método POST. El backend recibe la información, valida los campos, consulta la base de datos y genera una respuesta según el resultado obtenido.

Cuando las credenciales son correctas, el servidor crea una sesión autenticada. Posteriormente, el navegador puede solicitar recursos protegidos, como el dashboard o el módulo de conductores. Antes de devolver la información, el backend verifica que exista una sesión válida.

En una arquitectura basada en servicios, el frontend también podría comunicarse mediante una API REST. En este caso, el backend recibiría y devolvería información en formato JSON. Las solicitudes que modifiquen datos deberían incluir mecanismos de autenticación, validación y protección CSRF según la estrategia de seguridad utilizada.

5.5. Flujo de la primera petición autenticada

La Figura 12 representa el proceso ejecutado desde que el usuario ingresa sus credenciales hasta que visualiza una página protegida.

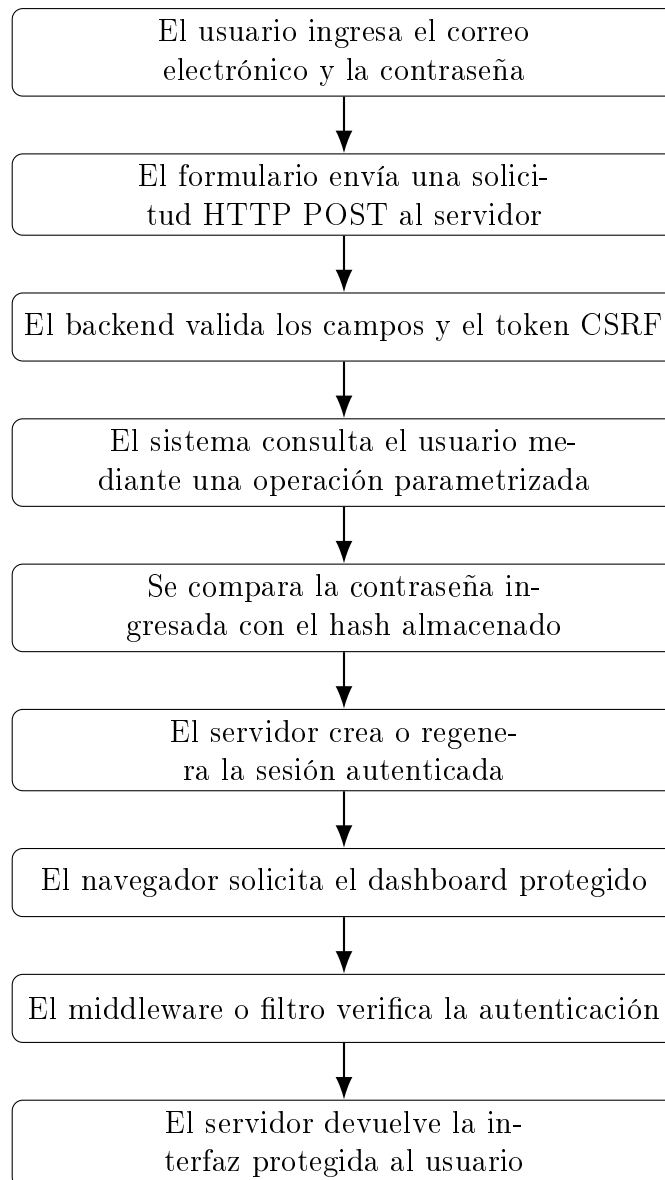


Figura 12: Flujo de la primera petición autenticada en SGROAS.

El proceso es semejante en las dos tecnologías. En PHP, la verificación se realiza mediante sesiones y funciones desarrolladas dentro del proyecto. En Spring Boot, la solicitud atraviesa los filtros configurados por Spring Security antes de permitir el acceso al recurso protegido.

6. Conclusiones

El desarrollo de SGROAS permitió aplicar los principales conceptos relacionados con la programación web del lado del servidor. Se implementaron dos versiones funcionales del sistema utilizando PHP y Java con Spring Boot. Ambas aplicaciones incorporaron registro de usuarios, inicio y cierre de sesión, protección de rutas y un módulo CRUD para administrar la información de los conductores.

La aplicación de controles de seguridad permitió fortalecer el tratamiento de la información. El saneamiento de las salidas contribuyó a prevenir ataques XSS, mientras que los tokens CSRF protegieron los formularios frente a solicitudes no autorizadas. De igual manera, el almacenamiento de contraseñas mediante algoritmos de hash evitó conservar credenciales en texto plano. Las consultas preparadas y los repositorios ayudaron a reducir los riesgos de inyección SQL.

La comparación realizada permitió determinar que PHP facilita un desarrollo más directo y requiere una menor configuración inicial. Sin embargo, diferentes mecanismos de protección deben implementarse manualmente. Spring Boot necesita una estructura más amplia, pero proporciona componentes integrados para la autenticación, la autorización, la protección CSRF, las cabeceras HTTP y el acceso organizado a los datos.

Finalmente, se comprobó que las dos tecnologías pueden utilizarse para construir aplicaciones web dinámicas y seguras. La elección depende del alcance del proyecto, la experiencia del equipo y los recursos disponibles. PHP resulta apropiado para sistemas de tamaño pequeño o mediano, mientras que Spring Boot ofrece ventajas para aplicaciones que requieren una arquitectura más organizada, escalabilidad y mantenimiento a largo plazo.

7. Repositorio del proyecto

<https://github.com/Alxjandr07/-GRUPO-E-PRACTICA-EXPERIMENTAL-UNIDAD-2>.
git

Referencias bibliográficas en formato IEEE

- [1] PHP Documentation Group, “PDO::prepare – Prepares a statement for execution,” *PHP Manual*. [En línea]. Disponible en: <https://www.php.net/manual/en/pdo.prepare.php>. [Consultado: 20-jun-2026].
- [2] PHP Documentation Group, “password_hash – Creates a password hash,” *PHP Manual*. [En línea]. Disponible en: <https://www.php.net/manual/en/function.password-hash.php>. [Consultado: 20-jun-2026].
- [3] PHP Documentation Group, “password_verify – Verifies that a password matches a hash,” *PHP Manual*. [En línea]. Disponible en: <https://www.php.net/manual/en/function.password-verify.php>. [Consultado: 20-jun-2026].
- [4] Spring, “Spring Security Reference,” *Spring Documentation*. [En línea]. Disponible en: <https://docs.spring.io/spring-security/reference/index.html>. [Consultado: 20-jun-2026].
- [5] Spring, “Cross Site Request Forgery,” *Spring Security Reference*. [En línea]. Disponible en: <https://docs.spring.io/spring-security/reference/servlet/exploits/csrf.html>. [Consultado: 20-jun-2026].
- [6] Spring, “Password Storage,” *Spring Security Reference*. [En línea]. Disponible en: <https://docs.spring.io/spring-security/reference/features/authentication/password-storage.html>. [Consultado: 20-jun-2026].
- [7] Spring, “Security HTTP Response Headers,” *Spring Security Reference*. [En línea]. Disponible en: <https://docs.spring.io/spring-security/reference/servlet/exploits/headers.html>. [Consultado: 20-jun-2026].
- [8] Spring, “Spring Data JPA Reference Documentation,” *Spring Data*. [En línea]. Disponible en: <https://docs.spring.io/spring-data/jpa/reference/index.html>. [Consultado: 20-jun-2026].
- [9] Thymeleaf, “Tutorial: Using Thymeleaf,” *Thymeleaf Documentation*. [En línea]. Disponible en: <https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html>. [Consultado: 20-jun-2026].
- [10] OWASP Foundation, “OWASP Top 10:2021,” *Open Worldwide Application Security Project*. [En línea]. Disponible en: <https://owasp.org/Top10/2021/>. [Consultado: 20-jun-2026].
- [11] OWASP Foundation, “Cross Site Scripting Prevention Cheat Sheet,” *OWASP Cheat Sheet Series*. [En línea]. Disponible en: <https://cheatsheetseries.owasp.org/>

`cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html`. [Consultado: 20-jun-2026].

- [12] OWASP Foundation, “Cross-Site Request Forgery Prevention Cheat Sheet,” *OWASP Cheat Sheet Series*. [En línea]. Disponible en: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html. [Consultado: 20-jun-2026].
- [13] OWASP Foundation, “SQL Injection Prevention Cheat Sheet,” *OWASP Cheat Sheet Series*. [En línea]. Disponible en: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html. [Consultado: 20-jun-2026].

Referencias bibliográficas en formato APA

- Eclipse Foundation. (s. f.-a). *Jakarta Server Pages specification*. Jakarta EE. <https://jakarta.ee/specifications/pages/>
- Eclipse Foundation. (s. f.-b). *Jakarta Servlet specification*. Jakarta EE. <https://jakarta.ee/specifications/servlet/>
- Microsoft. (s. f.-a). *ASP.NET Core documentation*. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/>
- Microsoft. (s. f.-b). *ASP.NET Core fundamentals overview*. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/>
- Microsoft. (s. f.-c). *Overview of ASP.NET Core MVC*. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/mvc/overview>
- Microsoft. (s. f.-d). *Razor Pages architecture and concepts in ASP.NET Core*. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/razor-pages/>
- Murach, J., & Harris, R. (2022). *Murach's PHP and MySQL* (4.^a ed.). Mike Murach & Associates.
- Oracle. (s. f.). *Java Platform, Standard Edition documentation*. <https://docs.oracle.com/en/java/javase/>
- Perl 5 Porters. (s. f.). *perldoc.perl.org: Official documentation for the Perl programming language*. <https://perldoc.perl.org/>
- Spring. (s. f.). *Spring Boot reference documentation*. <https://docs.spring.io/spring-boot/reference/index.html>
- The PHP Group. (s. f.). *PHP manual*. <https://www.php.net/manual/en/>
- Wall, L., Christiansen, T., & Orwant, J. (2000). *Programming Perl* (3.^a ed.). O'Reilly Media.